

Part 2 - RDDs

RDDs – Resilient Distributed Datasets

- **Immutable**
- **Fault-tolerant**
- **Distributed**
- New RDD created for each operation
- Automatically recovers from node failure via Lineage
- Different RDDs cached in different workers

When does the computations happen?

```
val textFile = sc.textFile("hdfs://...")  
val counts = textFile.flatMap(line => line.split(" "))  
                    .map(word => (word, 1))  
                    .reduceByKey(_ + _)  
counts.saveAsTextFile("hdfs://...")
```

Spark Operations and Lazy evaluation

Transformations (create new RDD)

Actions (Return a value)

- What is Lazy evaluation here?
- Where is Lazy evaluation vs. Materialised evaluation?

E.g. which one is lazy?

Groupby?

SaveAs?

Count?

Map?

Reduce?

Programming with RDDs

- SparkContext **sc** variable from Spark Shell to access functionalities
- Create RDD via Spark Contexts `sc.parallelize/textFile`
- Perform some transformations (create new RDD)
- Perform some actions (output results of operations on RDD)

Note on Pair-RDDs (**can** be used as key-value pairs)

RDD as key-value pairs: Example

```
pets = sc.parallelize([("cat", 1), ("dog", 1), ("cat", 2)])
```

```
pets2 = pets.reduceByKey((x, y) -> x + y)    // gives { (cat, 3), (dog, 1) }
```

```
pets3 = pets.groupByKey()                    // gives { (cat, [1, 2]), (dog, [1]) }
```

```
pets4 = pets.sortByKey()                     // gives { (cat, 1), (cat, 2), (dog, 1) }  
}
```

RDD as key-value pairs: Parallelism

```
pets = sc.parallelize([("cat", 1), ("dog", 1), ("cat", 2)])
```

```
pets2 = pets.reduceByKey((x, y) -> x + y, 5) // gives { (cat, 3), (dog, 1) }
```

```
pets3 = pets.groupByKey() // gives { (cat, [1, 2]), (dog, [1]) }
```

```
pets4 = pets.sortByKey() // gives { (cat, 1), (cat, 2), (dog, 1) }
```

RDD operation (non exhaustive list)

- map
- filter
- groupBy
- sort
- union
- join
- leftOuterJoin
- rightOuterJoin
- flatMap
- reduce
- count
- fold
- reduceByKey
- groupByKey
- cogroup
- cross
- zip
- cartesian
- sample
- take
- first
- partitionBy
- mapWith
- pipe
- distinct
- save

RDD operation (non exhaustive list)

transformation	description
map (<i>func</i>)	return a new distributed dataset formed by passing each element of the source through a function <i>func</i>
filter (<i>func</i>)	return a new dataset formed by selecting those elements of the source on which <i>func</i> returns true
flatMap (<i>func</i>)	similar to map, but each input item can be mapped to 0 or more output items (so <i>func</i> should return a Seq rather than a single item)
sample (<i>withReplacement</i> , <i>fraction</i> , <i>seed</i>)	sample a fraction <i>fraction</i> of the data, with or without replacement, using a given random number generator <i>seed</i>
union (<i>otherDataset</i>)	return a new dataset that contains the union of the elements in the source dataset and the argument
distinct ([<i>numTasks</i>]))	return a new dataset that contains the distinct elements of the source dataset

RDD operation (non exhaustive list)

transformation	description
groupByKey ([<i>numTasks</i>])	when called on a dataset of (K, V) pairs, returns a dataset of (K, Seq[V]) pairs
reduceByKey (<i>func</i> , [<i>numTasks</i>])	when called on a dataset of (K, V) pairs, returns a dataset of (K, V) pairs where the values for each key are aggregated using the given reduce function
sortByKey ([<i>ascending</i>], [<i>numTasks</i>])	when called on a dataset of (K, V) pairs where K implements Ordered, returns a dataset of (K, V) pairs sorted by keys in ascending or descending order, as specified in the boolean ascending argument
join (<i>otherDataset</i> , [<i>numTasks</i>])	when called on datasets of type (K, V) and (K, W), returns a dataset of (K, (V, W)) pairs with all pairs of elements for each key
cogroup (<i>otherDataset</i> , [<i>numTasks</i>])	when called on datasets of type (K, V) and (K, W), returns a dataset of (K, Seq[V], Seq[W]) tuples – also called <i>groupWith</i>
cartesian (<i>otherDataset</i>)	when called on datasets of types T and U, returns a dataset of (T, U) pairs (all pairs of elements)

End of Part 2 – RDDs